

# "잘 작동하는 RAG"를 어떻게 증명할 것인가

Session 2~3에서 RAG를 만들고 튜닝했다면, 이번은 측정 · 진화 · 운영이다.

Agentic RAG의 자기교정 루프, MCP 표준, RAGAS 4지표, Golden set 회귀 테스트, 그리고 Prompt Caching까지.

**34%→78%**

Agentic RAG CRAG 정확도  
(traditional vs agentic)

**-90%**

Prompt Caching  
입력 토큰 비용

**-85%**

Prompt Caching  
장문 응답 지연

**+27pt**

본 강의 MultiHopRAG 실측  
Hybrid → Self-RAG (acc)

0

### 왜 평가가 중요한가

"안 작동하는 RAG"의 침묵

1

### Agentic RAG

Planning · Self-correction · Loop

2

### Claude Tool Use

검색을 도구로 추상화

3

### MCP (Model Context Protocol)

$M \times N \rightarrow M+N$  통합

4

### RAGAS 4지표

Faithfulness · Relevancy · P/R

5

### Golden Set · LLM-as-Judge

회귀 테스트 만들기

6

### 운영 — 비용·지연·캐싱

Anthropic Prompt Caching

7

### 실측 파일럿 — MultiHopRAG

Agentic 5 config · RAGAS 4지표



### 10문제 퀴즈

Agentic · RAGAS · 운영 · self-check

# "안 작동하는 RAG"는 침묵한다 — 그래서 측정이 필수

Session 1~3에서 배운 모든 기법은 검증 없이 도입하면 의미가 없다. RAG는 틀린 답도 자신감 있게 만들어내고, 사용자는 그걸 알기 어렵다. Session 4의 핵심: 측정 가능한 시스템을 만들고, 그 위에서 진화시키기.

SESSION 4의 두 축 — AGENTIC (능동성) × EVALUATION (측정)

↑ Agentic (자율성)

**Agentic만 도입**

"멋있어 보이지만  
비용·지연만 폭증"

★ **둘 다 (Session 4 목표)**

측정→개선→측정 사이클  
Agentic을 데이터로 정당화

**기본 RAG, 평가 없음**

"잘 되는 것 같은데..."  
실패 모르고 운영

**평가만 도입**

"RAGAS 0.71에서  
멈춰버렸다"

→ Evaluation Maturity (측정 가능성)



## 측정 없는 개선은 미신

"이 기법이 더 좋다"는 감만으로 결정하면 반대 결과가 나와도 알 수 없다. Golden set + RAGAS + LLM-as-Judge로 수치로 비교해야 진짜 진화.



## Agentic은 도구지 목적이 아님

Self-RAG, ReAct 같은 패턴은 특정 문제(multi-hop, 자기교정)에 효과적. 모든 RAG를 agentic으로 바꿀 필요 없다. 적용 전 평가셋으로 ROI 측정.



## 운영 = 품질 × 비용 × 지연

Prompt caching, batching, 모델 라우팅으로 품질 유지하며 비용 90% 절감 가능. 측정 없이는 어디를 깎을지 모른다.

# Agentic RAG — 한 번 검색하지 않고, LLM이 검색을 "조종"

기존 RAG는 "질문 → 한 번 검색 → 답"의 정적 파이프라인. Agentic RAG는 LLM이 검색 도구를 직접 호출하고, 결과를 보고 다시 검색할지·답할지·재계획할지를 스스로 결정한다. 복잡한 질문에선 큰 차이가 나지만 비용·지연도 그만큼 커진다.

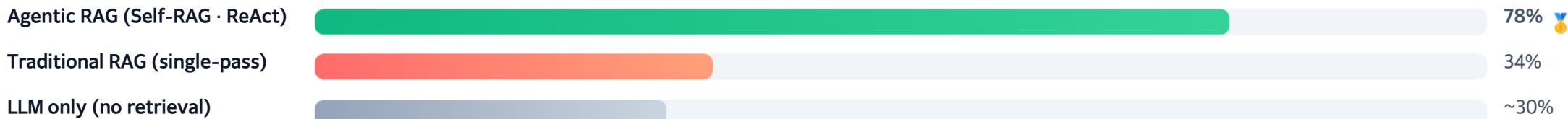
단일 RAG VS AGENTIC RAG — 같은 질문, 다른 흐름



## 단일 vs Agentic — 실측 차이

CRAG (Comprehensive RAG, Meta 2024) 벤치마크 — 복합 multi-hop 질문 위주

### CRAG 정확도 (복잡 multi-hop 질문)



### 단순 single-hop 질문에서는?

큰 차이 없음. Agentic는 단순 질문에선 같은 정답을 더 비싸게/느리게 만들 뿐. "이 질문에 agentic이 필요한가?"를 LLM이 첫 단계에서 판단하게 하는 게 핵심.

### Self-RAG의 환각률

2025 MDPI Electronics 연구 — 12개 RAG 변종 비교:

**Self-RAG: 환각률 5.8% (최저)**

표준 agentic 파이프라인: 12~14%

→ Self-RAG의 "검색 필요? 답이 근거 있나?" 자기교정 단계가 효과 있음.

### ⚠ 비용·지연 트레이드오프

Agentic은 LLM을 2~5번 호출한다 (planning + tool calls + synthesis).

지연: 단일 RAG 1~2초 → Agentic 3~10초.

비용: 단일 RAG \$0.001/쿼리 → Agentic \$0.005~0.02.

복잡 질문 비율이 높을 때만 ROI.

## 대표적인 Agentic RAG 패턴 4가지

### Self-RAG (Reflection)

매 단계에 "이 검색 결과가 충분한가?"를 LLM이 self-rate. 부족하면 재검색, 환각 의심되면 답변 폐기.

- ✓ 토큰 형태로 "[Retrieval Needed]", "[Supported]" 마킹
- ✓ 환각률 최저 (5.8%, 위 측정)
- ✓ 한 답변에 LLM 3~5번 호출

### ReAct (Reasoning + Acting)

Thought → Action → Observation → Thought ... 의 chain. LLM이 "생각"을 출력하고 도구 호출, 결과 보고 다음 생각.

- ✓ 모든 agent framework의 기반
- ✓ 도구 호출 형식이 명시적이라 디버깅 쉬움
- ✓ 긴 추론 시 토큰 누적

### CRAG (Corrective RAG)

검색 후 retrieval evaluator로 결과를 평가. 품질 낮으면 자동으로 web search·query rewrite 등 fallback.

- ✓ 실패 자동 복구
- ✓ knowledge graph 보강에 효과
- ✓ evaluator 학습 필요

### Plan-and-Execute

먼저 전체 계획을 세우고 (예: "1. 회사 매출 찾기 2. 직원수 찾기 3. 합치기"), 그 다음 각 단계 실행.

- ✓ 복잡 분석 질문에 효과
- ✓ 계획 잘못되면 전체 실패
- ✓ 병렬 실행 가능

구현 [LangGraph](#) 상태 머신 기반 agent (LangChain Inc)

[LlamaIndex](#) AgentRunner · ReActAgent · QueryEngineTool

[Claude SDK](#) Anthropic tool use loop (네이티브)

커스텀 while loop + Claude tool calling으로 충분

 **실무 판단 기준** — 질문의 30%+ 가 multi-hop·복합·반복적이면 Agentic. 그 미만이면 단일 RAG + Query Decomposition으로 충분. Agentic 도입 전 평가셋에 "복잡 질문" 라벨링 필수.

# Claude Tool Use — 검색을 "도구"로 호출

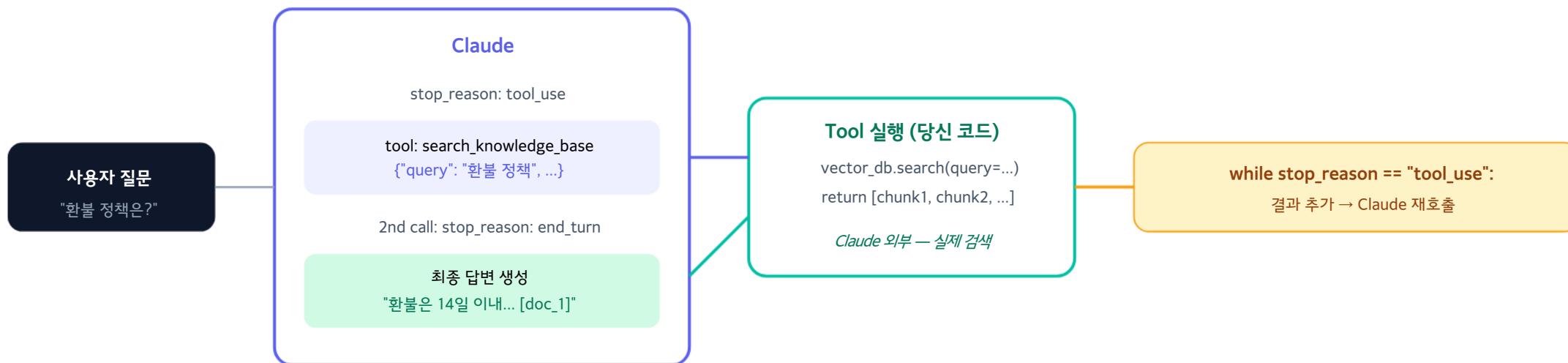
Agentic RAG의 가장 깔끔한 구현 방법. 검색기를 함수로 만들어 JSON Schema로 Claude에 알려주고, Claude가 필요할 때 자율적으로 호출하게 한다. 별도 프레임워크 없이도 가능.

## 📖 Tool 정의 — JSON Schema 형식

Claude에게 "이런 검색 도구가 있어, 이렇게 호출해" 라고 알려주는 스펙.

```
# 검색을 도구로 정의
tools = [
  {
    "name": "search_knowledge_base",
    "description": "사내 위키와 정책 문서를 검색합니다. 질문이 회사 내부 정보를 요구할 때 사용.",
    "input_schema": {
      "type": "object",
      "properties": {
        "query": {
          "type": "string",
          "description": "검색 쿼리. 검색 인덱스에 최적화된 표현으로."
        },
        "category": {
          "type": "string",
          "enum": ["HR", "엔지니어링", "재무", "법무"],
          "description": "질문이 어느 카테고리에 속하는지 메타필터"
        },
        "top_k": {
          "type": "integer",
          "default": 5
        }
      },
      "required": ["query"]
    }
  }
]
```

## Agent Loop — 호출 사이클



*# Agent Loop — Python pseudo-code*

```
messages = [{"role": "user", "content": question}]
```

```
while True:
```

```
    response = client.messages.create(  
        model="claude-sonnet-4-6",  
        tools=tools,  
        messages=messages  
    )
```

```
    if response.stop_reason == "end_turn":
```

```
        break # 최종 답변
```

```
    if response.stop_reason == "tool_use":
```

```
        for block in response.content:
```

```
            if block.type == "tool_use":
```

```
                result = execute_tool(block.name, block.input)
```

```
                messages.append({"role": "assistant", "content": response.content})
```

```
                messages.append(  
                    {"role": "user",  
                     "content": [{"type": "tool_result", "tool_use_id": block.id, "content": result}]}  
                )
```

## 왜 Tool로 호출하는 게 더 나은가

### 👍 도구 추상화의 이점

- ✓ LLM이 "검색이 필요한가" 자체를 판단 — 단순 질문에 검색 안 함 (비용↓)
- ✓ 여러 검색기 호출 가능 — 벡터, BM25, SQL, 캘린더 API 등 동시
- ✓ LLM이 query를 검색 인덱스에 맞게 자동 변형 (rewriting 내장)
- ✓ 도구 결과를 보고 재호출 (자기교정)

### ⚠️ 주의사항

- ✗ LLM이 도구 호출 안 하고 자기 지식으로 답할 위험 → 프롬프트로 강제
- ✗ 잘못된 인자 전달 시 디버깅 어려움 → 입력 validation 추가
- ✗ 무한 루프 방지 → max\_iterations 강제 (보통 5~10)
- ✗ 병렬 호출 사용 — Claude는 한 응답에서 여러 도구 호출 가능

구현

Claude API tools 파라미터 + tool\_use stop\_reason

LangChain @tool 데코레이터 + create\_tool\_calling\_agent

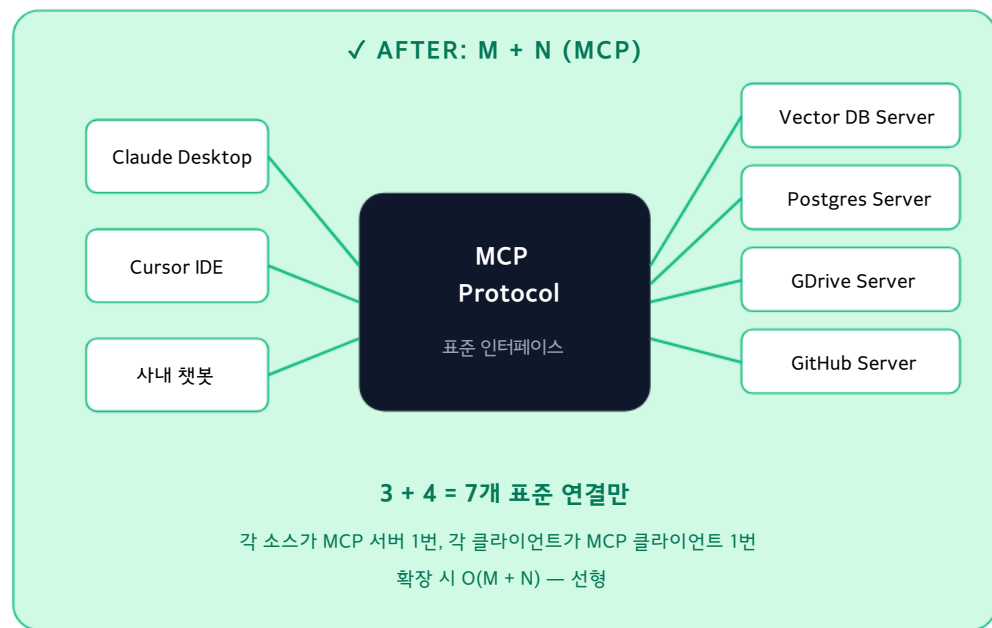
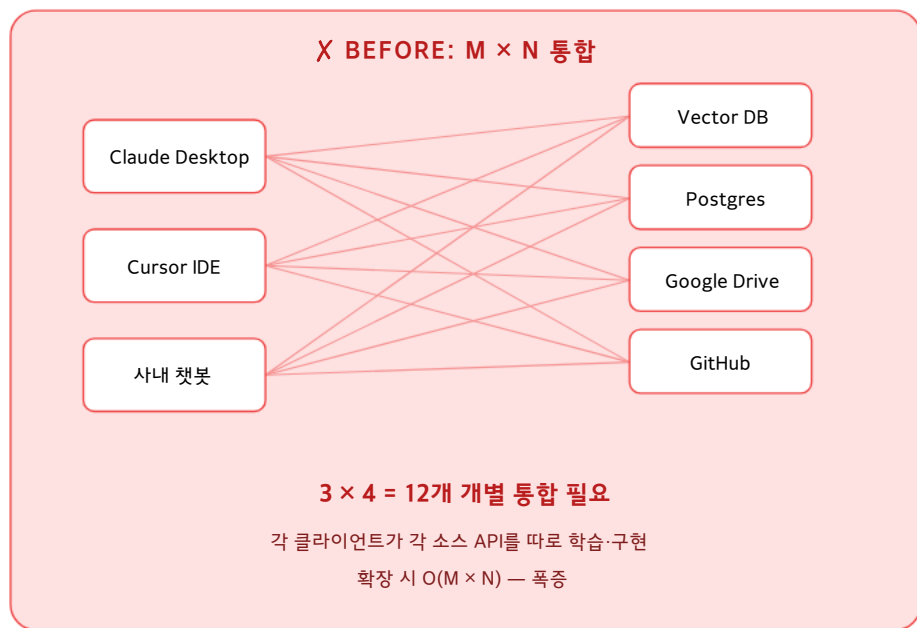
LlamaIndex FunctionTool · QueryEngineTool

# MCP — AI와 데이터 소스를 잇는 USB-C

MCP(Model Context Protocol)는 2024년 11월 Anthropic이 발표한 오픈 표준이다. AI 앱이 데이터·도구·서비스에 연결되는 방식을 통일해서,  $M$ 개 클라이언트  $\times$   $N$ 개 데이터소스 =  $M \times N$  통합 작업을  $M+N$ 으로 줄인다.

RAG 관점에선 "검색기를 표준 인터페이스로 노출"하는 방법.

왜 MCP인가 —  $M \times N$  문제와  $M+N$  해법



## MCP 아키텍처 — Host · Client · Server

### Host

AI 앱 그 자체. Claude Desktop, Cursor, Cline 등.  
사용자와 직접 상호작용하며 LLM을 호출.

### Client

Host 안에 있는 connection handler. 1개 MCP Server에  
1대1로 연결. Server의 도구·리소스·프롬프트 목록을  
LLM에 노출.

### Server

데이터 소스를 MCP 표준으로 노출하는 프로세스. 도구  
(tools), 리소스(resources), 프롬프트(prompts)를 제공.

## MCP와 RAG의 만남

전통적 RAG는 검색기를 코드 안에 내장한다. MCP RAG는 **검색을 외부 서버로 분리**해 모든 AI 앱이 같은 검색기를 쓰게 한다.

```
# MCP Server 예시 (Python SDK)
from mcp.server import Server
from mcp.types import Tool

app = Server("company-knowledge")

@app.list_tools()
async def list_tools():
    return [
        Tool(
            name="search_docs",
            description="회사 문서·정책 검색",
            inputSchema={ ... }
        ),
        Tool(name="search_code", description="코드베이스 검색", ...),
    ]

@app.call_tool()
async def call_tool(name, args):
    if name == "search_docs":
        results = vector_db.search(args["query"], top_k=5)
        return [format_chunk(r) for r in results]
    ...
```

### 👍 MCP RAG의 장점

- ✓ 한 RAG 서버 → N개 AI 클라이언트 재사용
- ✓ 도구 추가·교체가 클라이언트 코드 무변경
- ✓ 권한·인증을 서버 단에서 일관 관리
- ✓ Claude Desktop·Cursor 등에서 즉시 사용 가능

### ⚠️ 한계·고려

- ✗ 아직 표준 초기 — spec 변경 가능성 (2024-11 발표)
- ✗ 서버 실행 관리 필요 (프로세스·containerization)
- ✗ 네트워크 호출 오버헤드 (수십 ms)
- ✗ 레거시 단일 RAG 마이그레이션 비용

구현

MCP Python SDK [modelcontextprotocol/python-sdk](https://github.com/modelcontextprotocol/python-sdk)

MCP TypeScript SDK [modelcontextprotocol/typescript-sdk](https://github.com/modelcontextprotocol/typescript-sdk)

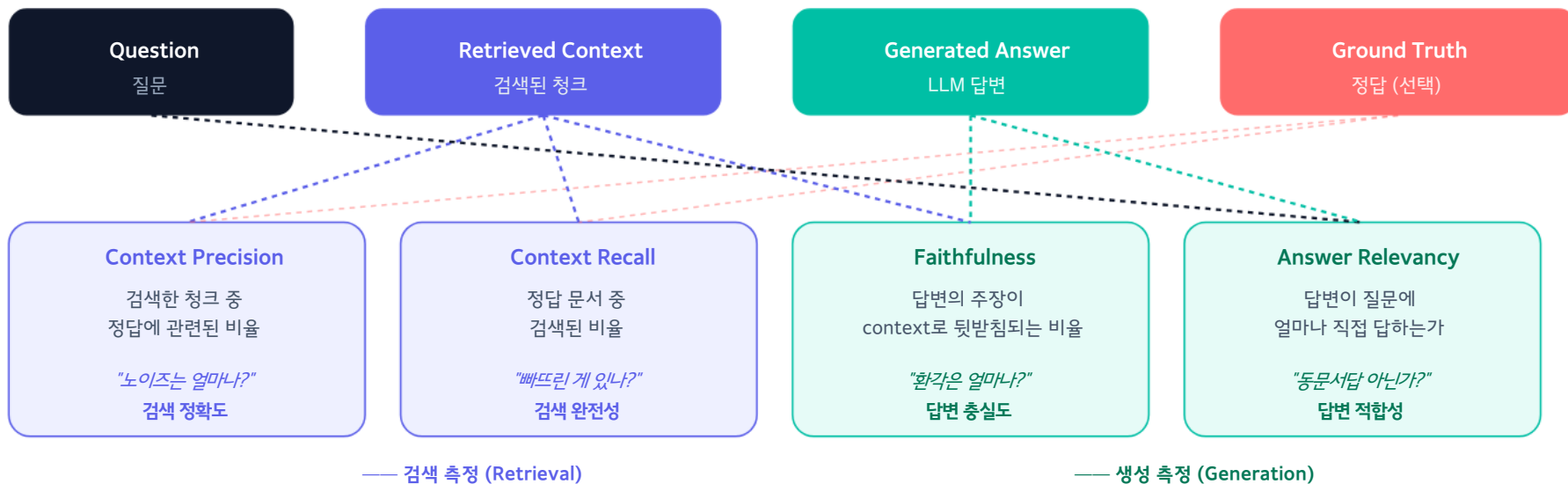
공식 서버 [filesystem](#) · [git](#) · [postgres](#) · [gdrive](#) · [brave-search](#)

🎯 언제 도입할까 — 팀이 여러 AI 앱(Claude Desktop, Cursor, 자체 챗봇)을 동시에 운영하면 MCP 진가 발휘. 단일 챗봇만 만들면 Tool Use(앞 섹션)로 충분. 2025년 표준으로 자리잡는 추세이므로 신규 프로젝트는 MCP 호환 고려.

# RAGAS — RAG를 4가지 지표로 측정

RAG는 두 가지 일을 한다: ① 좋은 청크 찾기 (검색)와 ② 좋은 답변 만들기 (생성). RAGAS는 이 둘을 각 2개씩, 총 4개 지표로 측정한다. LLM-as-Judge 방식이라 정답 라벨 없이도 자동 측정 가능.

4개 지표의 역할 — 검색과 생성을 각각 측정



## 4지표 — 공식과 직관

검색 정확도

### 1. Context Precision (CP)

검색한 청크들 중 정말로 정답을 만드는 데 도움 되는 청크의 비율. Top-K에 노이즈 청크가 많을수록 낮음. 1.0이면 모든 청크가 유용.

$$CP@K = \sum_{k=1..K} (\text{precision}@k \times \text{relevance}@k) / |\text{relevant items in Top-K}|$$

예시: Top-5 청크 중 3개가 정답에 직접 관련, 2개는 노이즈 → CP는  $3/5 = 0.60$  (LLM이 각 청크의 관련성을 판정)

검색 완전성

### 2. Context Recall (CR)

정답을 만드는 데 필요한 정보가 검색된 청크 안에 다 들어있는지. ground truth 답변을 문장 단위로 쪼개, 각 문장이 청크에 있는지 LLM으로 검증.

$$CR = (\text{Ground Truth 문장 중 context로 attributable한 비율})$$

예시: ground truth가 4문장인데 그 중 3문장만 검색된 청크에서 발견 →  $CR = 3/4 = 0.75$ . 검색 단계가 빠뜨린 정보 있음을 의미.

답변 충실도

### 3. Faithfulness

LLM 답변의 각 주장이 검색된 청크로 뒷받침되는가. 답변을 claim 단위로 쪼개, 각 claim이 context에서 추론 가능한지 LLM이 판정. 환각 직접 측정.

$$\text{Faithfulness} = (\text{Context로 뒷받침되는 claim 수}) / (\text{전체 claim 수})$$

예시: 답변에 5개 사실 주장, 4개는 청크에서 확인되지만 1개는 LLM이 지어낸 것 → 0.80. 환각이 있다는 신호.

답변 적합성

### 4. Answer Relevancy

답변이 질문에 직접 답하는가, 동문서답이 아닌가. LLM이 답변을 보고 가능한 질문들을 역추론하고, 원래 질문과 임베딩 유사도를 계산.

$$\text{Relevancy} = \text{mean}(\text{cosine\_sim}(\text{original\_q}, \text{reverse\_generated\_q}))$$

예시: "환불 어떻게?"에 "회사 역사는..."으로 답하면 → 역추론된 가능 질문이 "회사 역사는?"이라 원 질문과 거리가 멀어 낮은 점수.

📖 어떻게 읽나 — 진단 사례

| 점수 패턴                          | 의미                 | 처방                              |
|--------------------------------|--------------------|---------------------------------|
| CP 낮음 + CR 정상                  | 검색이 노이즈 많이 가져옴     | Reranker 추가, Top-K 줄이기          |
| CR 낮음 + CP 정상                  | 검색이 정답을 못 찾음       | 임베딩 모델 교체, 청킹 점검, Hybrid 검색     |
| Faithfulness 낮음 + Context 정상   | LLM이 환각 — 컨텍스트 무시  | 프롬프트 강화 ("context만 사용"), 더 큰 모델 |
| Relevancy 낮음 + Faithfulness 정상 | 관련 없는 사실로 답 — 동문서답 | 프롬프트로 "질문에 직접 답하라" 강조           |
| 4지표 모두 낮음                      | RAG 자체 부실          | 임베딩·청킹·검색 처음부터 재설계              |
| 4지표 모두 0.8+                    | 운영 적합 수준           | 비용·지연 최적화로 이동                   |

구현

**RAGAS** `pip install ragas · evaluate(dataset, metrics=[...])`

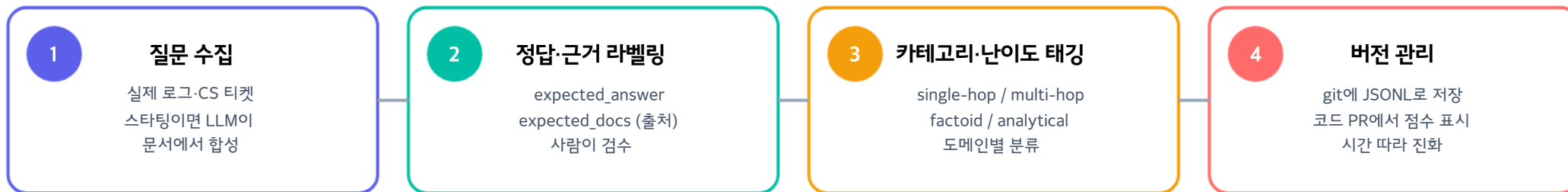
**유사 도구** DeepEval · TruLens · Phoenix (Arize) · Promptfoo

**판정 LLM** GPT-4 · Claude · Llama 등 어떤 LLM도 가능

# Golden Set 만들기 — 평가의 기반

RAGAS는 평가 도구지만, 무엇을 평가할지는 당신이 정해야 한다. **Golden set** = 사용자가 실제로 던질 만한 질문들 + (선택) 정답 + 출처 라벨. 이 세트가 좋아야 모든 측정이 의미 있다.

## Golden Set 만들기 — 4단계



### 크기 가이드

- ✓ 50개 — 절대 최소. 운영 가능성 확인 정도
- ✓ 200~500개 — 실용적 최소. 카테고리별 비교 가능
- ✓ 1,000~5,000개 — 견고. 회귀 테스트에 적합
- ✓ 너무 많으면? 평가 비용·시간 폭증 (LLM as Judge 호출 N배)

### 다양성 원칙

- ✓ 난이도 분포 (쉬움 50% / 중간 35% / 어려움 15%)
- ✓ 카테고리 균등 (HR · 엔지 · 재무 · 법무 ...)
- ✓ edge case 포함 ("정보 없음" 답해야 하는 질문)
- ✓ multi-hop 비율 측정 (10~30%)
- ✓ "context에 없음" 케이스도 — 환각 측정용

### LLM으로 합성 OK?

- ✓ 초기 부트스트랩엔 OK — 문서 → LLM에 "이 문서로 답할 질문 5개 만들어"
- ✓ 그러나 LLM 합성 질문은 너무 직설적이라 실제 사용자 패턴과 다름
- ✓ 실제 사용자 로그가 쌓이면 무조건 그쪽으로 교체
- ✓ 합성 + 실측 혼합이 현실적

## LLM-as-Judge — 자동 평가의 핵심

RAGs·DeepEval 같은 도구는 모두 LLM이 평가자가 되어 점수를 매기는 패턴을 쓴다. 사람보다 빠르고 일관되지만 한계도 있다.

### G-EVAL 패턴 — CHAIN-OF-THOUGHT 평가

```
# G-Eval 스타일 — Faithfulness 평가 프롬프트
```

```
"""
```

당신은 RAG 시스템의 답변을 평가하는 평가자입니다.

[검색된 컨텍스트]

{context}

[질문]

{question}

[생성된 답변]

{answer}

다음 단계를 따라 평가하세요:

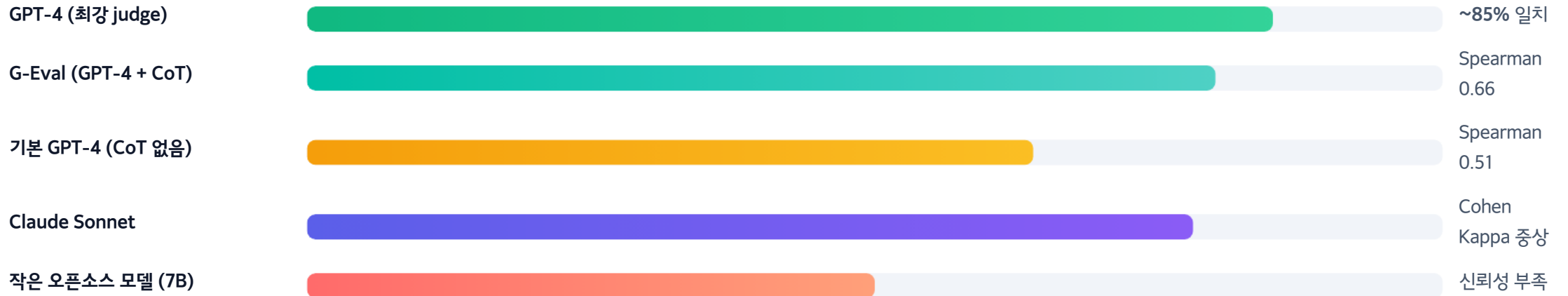
- 답변에 포함된 사실 주장(claim)을 모두 추출하세요.
- 각 claim이 [컨텍스트]에서 직접 추론될 수 있는지 yes/no로 판정하세요.
- (yes 개수) / (전체 claim 수)를 계산해 0~1 점수를 내세요.

다음 JSON 형식으로 응답:

```
{
  "claims": [
    {"claim": "환불은 14일 이내", "supported": true},
    {"claim": "ACME사 본사는 서울", "supported": false}
  ],
  "score": 0.5,
  "reasoning": "..."
}
```

```
"""
```

## LLM JUDGE의 인간 평가 일치율 — 어디까지 믿을 수 있나



arxiv 2510.09738 (Judge's Verdict, 2025), G-Eval 원논문, Snowflake RAG triad 벤치마크. 강력한 LLM(GPT-4/Claude) + Chain-of-Thought가 사실상 표준.

### 👍 LLM-as-Judge의 장점

- ✓ 대규모·일관 — 1000개 평가도 1시간이면 끝
- ✓ 이유와 함께 (설명 가능)
- ✓ 새 지표 추가가 프롬프트 한 줄
- ✓ 다국어·도메인 전이 자연스러움

### ⚠️ 알려진 편향

- ✗ Position bias — 먼저 나온 답을 선호
- ✗ Verbosity bias — 긴 답을 좋다 평가
- ✗ Self-preference — 같은 모델이 만든 답 선호
- ✗ 해결책: 위치 무작위화, 답변 길이 제어, 다른 모델로 judge

## 회귀 테스트 — RAG에 CI 붙이기

코드 PR마다 Golden set으로 자동 평가해 점수 하락을 막는다. 일반 unit test의 RAG 버전.

```
# .github/workflows/rag-eval.yml (예시)
name: RAG Evaluation
on: [pull_request]

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - name: Run RAGAS on golden set
        run: |
          python evaluate.py \
            --golden-set tests/golden_set.jsonl \
            --output results.json \
            --baseline main_branch_scores.json
      - name: Comment PR with scores
        uses: actions/github-script@v7
        with:
          script: |
            // faithfulness 5%p 이상 하락이면 fail
            if (delta.faithfulness < -0.05) core.setFailed("품질 회귀");
```

### 측정 빈도

매 PR — 핵심 50~100 골든 (빠른 피드백)  
매 일/주 — 전체 1000~5000 골든  
매 월 — 인간 검수 샘플링 (LLM judge 검증)

### 임계값 설정

Faithfulness  $\geq 0.90$  (환각 거의 없음)  
Context Recall  $\geq 0.85$   
Answer Relevancy  $\geq 0.85$   
하락 -3%p 이상이면 PR block

### 시간 따른 추적

대시보드(Grafana·Streamlit)에 점수 시계열 표시.  
모델·임베딩·칭킹 변경 시점 마킹.  
장기 트렌드로 데이터 drift 감지.

 **최소 시작 셋업** — Golden set 100개 (실제 사용자 질문 + 합성 혼합) + RAGAS 4지표 + Claude/GPT-4 as judge + Grafana 대시보드. 이것만 있어도 "감"으로 운영하던 RAG가 "측정 기반"으로 진화. 일주일 안에 구축 가능.

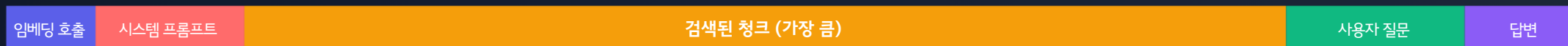
# 운영 — 비용 · 지연 · 캐싱

RAG가 잘 작동해도 한 쿼리에 \$0.1, 응답 10초면 사용자는 떠난다. Anthropic Prompt Caching, 모델 라우팅, 배치, 인덱스 최적화로 품질 유지하며 비용 90% / 지연 85% 절감이 가능하다.

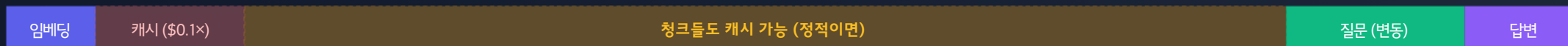
## 💰 어디에 돈이 나가는가

한 RAG 쿼리의 비용 분해 (CLAUDE SONNET 4.6 기준)

기본 단일 RAG (캐싱 없음)



Prompt Caching 적용 (90% 할인)



정적 부분(시스템 프롬프트, 자주 쓰는 청크 묶음)을 캐시 → 입력 비용의 90% 절감

## ⚡ Anthropic Prompt Caching — 핵심 무기

### 💰 가격 구조 (Claude API)

|                   |                  |
|-------------------|------------------|
| 기본 input          | 1.0×             |
| Cache write (5분)  | 1.25×            |
| Cache write (1시간) | 2.0×             |
| Cache read        | 0.1× (-90%) 🏆    |
| Output (변함 없음)    | 1.0× output rate |

손익분기: 5분 캐시는 1번만 재읽기해도 본전, 1시간 캐시는 2번이면 본전.

### 🕒 지연 절감

Anthropic 자체 보고:

**최대 -85%** latency reduction

실측 예시: 100K 토큰 책 입력

Before: 11.5초 → After: 2.4초

긴 시스템 프롬프트·문서가 캐시되면 매 호출마다 그걸 다시 처리 안 함 → TTFT(first token time) 크게 단축.

## 🎯 RAG에서 무엇을 캐시할까

### ✓ 캐시하기 좋은 것

#### 📄 System prompt (긴 instruction)

규칙·에시·페르소나 — 잘 안 바뀜

#### 📖 자주 쓰이는 documents

회사 정책·매뉴얼 — 정적 콘텐츠

#### 🔧 Tool 정의 (스키마들)

변할 일 없는 JSON Schema

#### 💬 대화 history (장기 세션)

multi-turn 챗봇 — 누적 대화

### X 캐시 효과 없거나 NG

#### ❌ 사용자 질문 그 자체

매번 다름 — 캐시 안 됨

#### ❌ 매번 검색되는 Top-K 청크

쿼리별로 다른 청크 → 캐시 miss

#### ⚠️ 1024 토큰 미만 콘텐츠

Sonnet 기준 최소 1024 토큰 — 작으면 비효율

#### ⚠️ 5분 안에 안 재호출되면

캐시 write 1.25x → 손해



## 실제 비용 계산 예시

시나리오: 사내 챗봇, Claude Sonnet 4.6, 시스템 프롬프트 5K 토큰, 사용자 질문 50토큰, 검색된 청크 3K 토큰, 답변 500 토큰. 시간당 1000 쿼리.

### 캐싱 없음

입력 토큰:  $5000 + 50 + 3000 = 8,050$

출력 토큰: 500

쿼리당 비용 (Sonnet \$3/M in, \$15/M out):

Input:  $8050 \times \$3/M = \$0.0242$

Output:  $500 \times \$15/M = \$0.0075$

합계: **\$0.0317 / query**

시간당 1000 쿼리:

→ **\$31.70 / hour**

→ **\$760 / day**

### + Prompt Caching (시스템 프롬프트 캐시)

5000 토큰을 캐시에 (write 한 번)

이후 cache read:  $5000 \times 0.1\times$

쿼리당 비용:

첫 호출:  $5000 \times \$3.75/M = \$0.0188$  (write)

+  $3050 \times \$3/M = \$0.0092$

+  $500 \times \$15/M = \$0.0075$

= **\$0.0355**

2번째 이후:  $5000 \times \$0.30/M = \$0.0015$  (read)

+  $3050 \times \$3/M = \$0.0092$

+  $500 \times \$15/M = \$0.0075$

= **\$0.0182**

1000 쿼리 평균: **~\$0.0182**

→ **\$18.20 / hour** (\$13.50 절약, -43%)

→ **월 ~\$10K 절약**. 청크 묶음까지 캐시할 수 있으면 추가로 -50% 가능.

## 🔧 비용·지연 최적화 — 다른 무기들

### 🚦 모델 라우팅

간단 질문 → Haiku(\$0.25/M),  
복잡 질문 → Sonnet(\$3/M),  
초복잡 → Opus(\$15/M).  
라우터 LLM이 첫 단계에서 분류.

### 📦 Batch API

대량 평가·인덱싱 작업은 **Batch API**로 (50% 할인).  
24시간 내 처리. 실시간 응답 아닐 때 활용.

### 📄 Context Compression

검색된 청크에서 관련 문장만 추출 후 LLM에.  
토큰 30~50% 절약. LangChain

`ContextualCompressionRetriever`

### 🕒 Async / Streaming

검색·임베딩·LLM 호출을 병렬로.  
Streaming으로 첫 토큰부터 표시 — 체감 지연 70%↓.

### 🔍 인덱스 튜닝

HNSW M·ef\_construct·ef\_search 조절로  
latency 5~10배 차이. RAM에 올라가는지 확인.

### 📊 모니터링

쿼리별 token usage·latency·RAGAS 점수를 함께 추적.  
Datadog · LangSmith · Phoenix · Helicone 등.

### 🎯 최소 운영 체크리스트

- ✓ **Prompt caching** 켜기 — 시스템 프롬프트만이라도 (1줄 설정으로 -40%)
- ✓ **Token usage 모니터링** — 어디서 새는지 알아야 깎을 수 있음
- ✓ **모델 라우팅 1단계** — 단순 질문 Haiku, 복잡 Sonnet
- ✓ **Streaming 응답** — 체감 지연 큰 폭 절감
- ✓ **RAGAS 점수와 비용 함께 추적** — 비용 깎으면서 품질도 보야

# 이 강의를 위해 직접 측정 — Agentic RAG와 RAGAS 4지표

§1 Self-RAG·Plan-and-Execute의 효과와 비용, §4 RAGAS 4지표의 진단력을 MultiHopRAG (Tang & Yang COLM 2024) 데이터셋에서 직접 측정했다. Session 3 §7의 baseline(dense/hybrid/decomp)에 agentic 2종을 더한 5 config 비교.

## 실험 셋업

- ✓ 코퍼스 609 뉴스 기사, stratified 250 Q (comparison 84·inference 80·temporal 57·null 29) — accuracy/latency용
- ✓ RAGAS 4지표: 같은 분포 stratified 120 Q 하위샘플 (모든 config에 동일한 질문 적용)
- ✓ Generator·Decomposer·Judge·RAGAS judge 모두 Claude Haiku 4.5 (local CLI pool), 임베딩 BGE-small-en-v1.5
- ✓ Self-RAG: max\_hops=3, 매 hop에 sufficient 자체평가 → 부족 시 다음 검색 쿼리 생성
- ✓ Plan-and-Execute: 첫 호출에 plan(1~4 sub-query) 생성 → 각 검색 RRF 합성 → 한 번에 답변

 결과 ① — 5 config 정확도 · 비용 · 지연

| 구성                                                             | 전체 acc              | Recall@6 | MRR   | Comparison<br>(n=84) | Inference<br>(n=80) | Temporal<br>(n=57) | Null<br>(n=29) | LLM 호출/Q | 지연/Q  |
|----------------------------------------------------------------|---------------------|----------|-------|----------------------|---------------------|--------------------|----------------|----------|-------|
| Baseline Dense<br><small>Session 3 §7</small>                  | 37.8%               | 52.1%    | 0.725 | 24.4%                | 40.0%               | 26.3%              | 93.1%          | 1.0      | —     |
| + Hybrid (BM25+Dense RRF)                                      | 50.6%               | 63.6%    | 0.779 | 36.3%                | 57.5%               | 38.6%              | 96.6%          | 1.0      | —     |
| + Query Decomposition                                          | 61.6%               | 70.8%    | 0.737 | 53.0%                | 70.0%               | 44.7%              | 96.6%          | 2.0      | —     |
| Agentic: Self-RAG<br><small>§1 적용 · iterative</small>          | 65.4%<br>⚠ null 19% | 70.6%    | 0.772 | 61.9%                | 90.6%<br>🏆          | 58.8%              | 19.0%          | 1.94     | 31.3s |
| Agentic: Plan-and-Execute<br><small>§1 적용 · plan-first</small> | 58.6%               | 69.5%    | 0.730 | 51.2%                | 58.8%               | 48.2%              | 100%           | 2.00     | 28.2s |

### 💡 Self-RAG의 Inference 90.6% — 강의 §1의 주장 재현

반복 검색이 가장 효과적인 질문 유형은 **inference**. Hop 1에서 부분 답을 얻고 → Hop 2에서 그것을 사용한 follow-up 검색 → 한 번 검색으로는 못 모으는 사실들을 누적한다. 평균 1.94 hops.

### ⚠️ Self-RAG의 Null 19% — Agentic의 환각 트레이드오프

Decomp는 null\_query에서 **96.6%** 거절했지만, Self-RAG는 **19%**로 무너졌다. Loop의 "마지막 hop에서 최선을 다해 답하라" 압력이 '모름'을 거절 못 하게 만든다. §1에서 본 agentic 환각 트레이드오프가 실제로 나타나는 모양.

### 💰 Agentic의 실질 비용 — LLM 호출 2배, 지연 30초+

Decomp 2 calls/Q (~0.5s) vs Self-RAG 1.94 calls/Q (**31.3s**) vs Plan-Execute 2.0 calls/Q (28.2s). Self-RAG·Plan-Execute의 latency가 큰 이유는 CLI subprocess 호출 자체의 overhead가 누적되기 때문. 프로덕션 API 호출이라도 **지연 3~10초**는 불가피.

### 🎯 Decomp가 sweet spot인 이유

Decomp는 acc 61.6%, calls 2.0, null 96.6%로 "**agentic의 정확도 + non-agentic의 안전성**"을 한 번에 가져온다. Self-RAG가 +3.8pt 더 정확하지만 그 대가는 null 거절률 -78pt. **실무 default는 여전히 decomp + hybrid**이며, agentic은 abstention 라우터를 추가로 둔 뒤 도입하는 것이 안전하다.

한 눈에 — 5 CONFIG 전체 정확도 (250-Q)



결과 ② — RAGAS 4지표 (5 config × 120 Q)

정답 라벨과 별개로, \$4 RAGAS 4지표가 같은 시스템을 어떻게 다르게 진단하는지.

| 구성                                   | 검색 정확도<br>Context Precision | 검색 완전성<br>Context Recall | 환각 ↓<br>Faithfulness | 동문서답 ↓<br>Answer Relevancy |
|--------------------------------------|-----------------------------|--------------------------|----------------------|----------------------------|
| Dense                                | 0.562                       | 0.564                    | 0.982                | 0.735                      |
| + Hybrid                             | 0.605 +0.04                 | 0.642 +0.08              | 0.958                | 0.762 +0.03                |
| + Decomp                             | 0.599                       | 0.678 +0.04              | 0.938                | 0.771                      |
| Self-RAG                             | 0.601                       | 0.758 🏆                  | 0.923 ⚠️             | 0.788 🏆                    |
| Plan-Execute<br><small>n=72*</small> | 0.741 🏆                     | 0.752                    | 0.953                | 0.773                      |

\* Plan-Execute RAGAS는 CLI 호출 실패로 일부 누락(120 → 72 측정). 다른 4 config는 119~120/120 채점 성공.

## 🔍 RAGAS가 보여주는 진단 — 강의 §4의 사례표 적용

### 📈 Recall이 retrieval 강화에 비례

Dense 0.564 → Hybrid 0.642 → Decomp 0.678 → Self-RAG 0.758 — **검색을 강화할수록 RAGAS Recall이 단조 증가**. 강의 §4 진단표의 "CR이 낮으면 임베딩 교체·칭킹 점검·Hybrid 검색"이 정확히 들어맞는 신호.

### 📉 Self-RAG: Faithfulness 단독 하락

Self-RAG의 Recall 0.758은 최고, AnsRel 0.788도 최고지만 **Faithfulness만 0.923으로 가장 낮음**. 강의 §4 진단표의 "Faithfulness 낮음 + Context 정상 = LLM 환각 (컨텍스트 무시)" 패턴이 정확히 나타난다. 250-Q acc의 null 19% 붕괴와 같은 원인 — agentic loop의 답 압력.

### 🎯 Plan-Execute: CP 최고 (0.741)

Plan으로 분해한 1~4 query 각각이 정확한 검색을 하니 **Top-K에 노이즈가 가장 적다**. 강의 §4 진단표의 "CP가 높으면 reranker 효용 ↓ — 이미 깨끗함"에 해당.

### ✓ 4지표의 가치 — accuracy 한 개로는 안 보이는 것

Self-RAG는 acc 1위(65.4%)지만 RAGAS는 "환각 위험 있음"을 정확히 잡아냈다. 반대로 Plan-Execute는 acc 4위(58.6%)지만 RAGAS는 "검색이 깨끗하고 답이 안전"하다고 보여준다. 운영에서 어느 것을 선택할지의 기준이 **acc만으로는 부족하다**는 §4 핵심 메시지.

 파일럿의 한계

- × 250 Q / 120 Q는 stratified subsample — full eval에서  $\pm 1 \sim 3$ pt 흔들릴 수 있음
- × RAGAS judge가 generator와 같은 모델 family(Haiku) — Sonnet judge로 재검 권장
- × Plan-Execute RAGAS 72/120만 성공 — CLI rate-limit 시점에 실패가 누적. acc 결과(250/250)는 영향 없음
- × Agentic은 본 셋업에서 CLI subprocess overhead가 latency를 부풀림 — 실제 API 호출이면 5~10초로 줄어듦

# Session 4 정리 — 측정 가능한 RAG의 완성

한 페이지 정리 — BUILD → MEASURE → IMPROVE → OPERATE

1

## BUILD

**Agentic RAG로 진화**  
Self-RAG · ReAct · CRAG  
Plan-and-Execute

**검색을 도구로 노출**  
Claude Tool Use API  
JSON Schema 기반

**MCP로 표준화**  
M+N 통합 패턴  
2024-11 Anthropic 발표

2

## MEASURE

**RAGAS 4지표**  
Context Precision/Recall  
Faithfulness  
Answer Relevancy

**Golden Set 만들기**  
200~500개 시작  
실제 + 합성 혼합

**LLM-as-Judge**  
GPT-4 / Claude  
인간 일치율 ~85%

3

## IMPROVE

**진단별 처방**  
CP↓ → Reranker 추가  
CR↓ → 임베딩·칭킹 점검  
Faith↓ → 프롬프트 강화

**회귀 테스트**  
매 PR마다 점수  
-3%p이면 block

**A/B 실험 문화**  
새 기법 도입 전 측정

4

## OPERATE

**Prompt Caching**  
입력 비용 -90%  
지연 -85%

**모델 라우팅**  
단순↔복잡 분리  
Haiku/Sonnet/Opus

**실시간 추적**  
token · latency · 품질  
LangSmith · Phoenix

 4 Session 통합 — RAG 운영 체크리스트

| Session                   | 핵심 결정                                                                      | 측정으로 확인                            |
|---------------------------|----------------------------------------------------------------------------|------------------------------------|
| Session 1<br>RAG 개념       | RAG가 필요한 시나리오인가? Fine-tuning·Long-context 대안 검토했나?                         | 사용자 질문 패턴 분석 · 문서 크기 측정            |
| Session 2<br>기초 부품        | 임베딩 (KURE-v1?) · 청킹 (Recursive 400자?) · 벡터 DB · 검색 (Hybrid?) · 프롬프트 (3원칙?) | Context Precision/Recall로 검색 품질 측정 |
| Session 3<br>고급 기법        | Reranker · Contextual Retrieval · Query Decomposition · Parent-Child를 어디서? | RAGAS 4지표 변화량으로 ROI 검증             |
| Session 4<br>Agentic + 평가 | Agentic 적용 시점? MCP 도입? Golden set 운영? Caching?                             | 회귀 테스트 + 비용/지연 대시보드                |

🎓 **4 Session 한 줄 요약** — Session 1·2: RAG 부품 조립 → Session 3: 검색 품질 끌어올리기 → Session 4: **측정 가능한 시스템**으로 운영. 마지막 단계가 빠지면 앞 3개도 헛수고다.

## 🚀 다음 단계 (Session 5+ 가능 주제)

### 🤖 Multi-modal RAG

이미지·표·차트가 포함된 PDF · 다이어그램. Voyage Multimodal, vision LLM 통합.

### 🕸 GraphRAG

엔티티·관계 그래프 + RAG. Microsoft GraphRAG, FalkorDB. multi-hop·도메인 QA에 강함.

### 🔒 보안·거버넌스

Row-level security, 사용자별 access control, audit log, PII 마스킹, prompt injection 방어.

# 10문제 퀴즈 — Agentic · RAGAS · 운영 self-check

Session 4의 7가지 주제(Agentic / Tool Use / MCP / RAGAS 4지표 / Golden Set / 운영 / \$7 실측)를 10문제로 정리. 각 문항 "정답 보기" 클릭 시 정답과 짧은 해설이 펼쳐진다.

 **풀이 방법** — 5지선다 또는 OX 형식. 해설에는 "왜 다른 보기는 틀린가"까지 포함. 8문제 이상이면 본 강의 졸업 — 실무 RAG를 측정·운영할 준비 완료.

1 CRAG 벤치마크에서 보고된 Agentic RAG vs Traditional RAG의 정확도 격차에 가장 근접한 값은?

A. 34% → 38% (+4pt)

B. 34% → 78% (+44pt)

C. 50% → 60% (+10pt)

D. 70% → 90% (+20pt)

정답 보기

**정답: B** 복합 multi-hop 질문에 한해서는 격차가 극단적이다. 단순 single-hop에서는 거의 차이 없음.

단순 질문에 Agentic을 쓰면 비싸기만 하다 — "이 질문에 agentic이 필요한가?"를 LLM이 첫 단계에서 판단하게 하는 게 실무 패턴.

## 2 Self-RAG 패턴의 핵심 작동 원리는?

- A. 검색 결과를 모두 합쳐 한 번에 LLM에 넣는다
- B. 매 단계에 "이 검색 결과가 충분한가?"를 LLM이 self-rate, 부족하면 재검색·환각 의심되면 답 폐기
- C. 외부 web search로 검색 결과를 보강한다
- D. 모든 청크에 신뢰도 점수를 미리 라벨링한다

정답 보기

**정답: B** Self-RAG은 token 형태로 "[Retrieval Needed]", "[Supported]" 같은 self-reflection을 출력해 자기교정한다.

2025 MDPI 연구에서 환각률 5.8%(외부 보고)로 최저였지만, 본 강의 §7 MultiHopRAG에서는 multi-hop 압력으로 null 거절률이 19%로 무너졌다 — 같은 메커니즘이 데이터셋에 따라 다르게 작용한다.

3 MCP(Model Context Protocol)가 풀려는 본질적 문제는?

A. LLM의 컨텍스트 윈도우 부족

B.  $M$ 개 LLM  $\times$   $N$ 개 도구 =  $M \times N$  통합 폭증  $\rightarrow M+N$ 으로 줄이기

C. RAG의 retrieval 정확도 부족

D. 임베딩 모델 표준화 부재

정답 보기

**정답: B** MCP는 "AI와 데이터 소스를 잇는 USB-C" — 각 LLM 클라이언트가 각 데이터/도구 서버에 표준 프로토콜로 연결되어 통합 작업을  $M \times N$ 에서  $M+N$ 으로 줄인다.

A·C·D는 MCP의 직접 해결 대상이 아님. MCP는 통합 비용 문제를 푼다.

4 RAGAS 4지표 중 검색 단계(Retrieval)를 측정하는 두 지표는?

A. Faithfulness · Answer Relevancy

B. Context Precision · Context Recall

C. Context Precision · Faithfulness

D. Context Recall · Answer Relevancy

정답 보기

**정답: B** CP(노이즈는 얼마나?) + CR(빠뜨린 게 있나?) = 검색 측정. Faithfulness(환각)와 Answer Relevancy(동문서답) = 생성 측정.

진단표: CP↓ CR정상 → reranker; CR↓ CP정상 → hybrid/임베딩 교체; Faith↓ → 프롬프트 강화; Rel↓ → "질문에 직접 답하라" 강조.

5 본 강의 §7 RAGAS 측정(120 Q)에서 Self-RAG의 Faithfulness가 단독으로 가장 낮게(0.92) 나온 원인은?

A. 검색 청크의 품질이 나빴다

B. RAGAS judge가 Self-RAG 답변을 잘 이해 못 했다

C. Self-RAG loop의 "마지막 hop에서 최선을 다해 답하라" 압력이 abstention을 무력화 — LLM이 context 밖 사실을 지어낸다

D. Top-K가 너무 컸다

정답 보기

**정답: C** 같은 패턴이 250-Q acc 측정의 null\_query 거절률 19% 붕괴(다른 config는 96%+)와도 일치 — 두 독립 측정이 같은 환각 원인을 가리킨다.

RAGAS의 가치 — acc만 보면 Self-RAG가 1위(65.4%)지만 Faithfulness 단독 저하가 "환각 위험"을 정확히 잡아낸다.

6 OX. Golden set은 "정답 라벨이 있는 평가 데이터"이며, 품질 좋은 50~250개가 형식적 1000개보다 RAG 운영에서 훨씬 가치 있다.

A. ☒ 맞다 (True)

B. ☐ 틀리다 (False)

정답 보기

**정답: A (True)** 도메인 전문가가 라벨링한 대표 질문 50~250개에 (질문 · 정답 · 답에 필요한 문서ID · 질문 유형 라벨)이 정확하게 들어있어야 측정이 의미 있다. 본 강의 §7 파일럿도 stratified 250 Q로 진행 — full eval 2,556 대비  $\pm 1 \sim 3$ pt jitter에 그친다.

"많기만 한 평가셋"은 라벨 노이즈가 측정값을 혼든다.

7 Anthropic Prompt Caching에서 RAG 시스템 중 캐시하기 가장 좋은 것은?

A. 사용자 질문 자체

B. 매번 검색되는 Top-K 청크

C. 긴 시스템 프롬프트 + 자주 쓰이는 정적 문서(회사 정책 등) + Tool 정의 스키마

D. 임베딩 벡터

정답 보기

**정답: C** Cache read는  $0.1\times$  (-90%) 비용. 잘 안 바뀌고 길고 모든 요청에 들어가는 것이 캐시 최적 대상. 손익분기: 5분 캐시는 1번 재읽기만 해도 본전, 1시간 캐시는 2번.

A·B는 매번 달라서 캐시 miss. D는 텍스트 토큰이 아니라 캐싱 대상 자체가 아님.

8 본 강의 §7 파일럿에서 측정된 Self-RAG의 질문당 평균 LLM 호출 횟수는?

A. 1.0

B. 1.5

C. 1.94 (평균 1~3 hop)

D. 3.5

E. 5.0+

정답 보기

**정답: C (1.94)** hop 분포: 1 hop 109개, 2 hop 48개, 3 hop 93개. 간단한 질문은 1 hop, 어려운 질문은 max\_hops(=3)까지 도달하는 동적 동작. Plan-Execute는 항상 2.0 calls/Q (plan + answer 2단계 고정).

CLI subprocess overhead 때문에 본 셋업의 latency는 30s대로 부풀려졌지만, native API는 3~10s 수준.

9 본 강의 §7 결과에 비추어, 실무 default로 추천되는 구성은?

A. Baseline Dense (가장 단순)

B. Self-RAG (가장 정확)

C. Hybrid + Decomposition (정확도와 안전성 균형)

D. Plan-and-Execute (가장 정확한 검색)

정답 보기

**정답: C** Decomp: acc 61.6%, calls/Q 2.0, null\_query 거절률 96.6%. "**agentic의 정확도 + non-agentic의 안전성**"을 한 번에 가져온다. Self-RAG가 +3.8pt 더 정확하지만 그 대가는 null 거절률 -78pt (96.6%→19%). 사용자에게 거짓을 답하는 위험.  
Agentic 도입 시에는 반드시 abstention 라우터를 따로 뒀야 안전.

10 RAG 시스템 도입 후 가장 먼저 측정해야 할 것은?

- A. 한 쿼리당 평균 토큰 비용
- B. 도메인 representative 질문 50~250개로 만든 평가셋의 acc / RAGAS
- C. 벡터 DB의 인덱스 빌드 시간
- D. 임베딩 모델의 MTEB 점수

정답 보기

**정답: B** "안 작동하는 RAG는 침묵한다" — 사용자가 잘못된 답을 봐도 모를 수 있다. 측정 없이 운영하는 RAG는 운영하지 않는 것과 같다. 평가셋이 있으면 매 변경마다 회귀 테스트가 가능 — 운영의 가장 큰 무기.

A·C·D는 RAG가 작동하기 시작한 후 최적화 단계에서 보는 것. 작동 자체를 증명하는 게 먼저.

### 🚩 스코어카드 — 본 강의 졸업 자격

- ✓ **9~10 정답** — RAG 시스템을 직접 설계·측정·운영할 준비 완료. 본 강의 졸업.
- ✓ **7~8 정답** — 핵심 이해 OK. 운영 시 RAGAS·Golden Set 부분을 다시 한번 참고.
- ✓ **5~6 정답** — Agentic·RAGAS·운영 중 약한 부분을 §1·§4·§6 다시 읽기.
- ✓ **4 이하** — Session 4를 처음부터 천천히. 특히 §1 agentic 패턴과 §4 RAGAS 진단표.

🎓 **전체 강의 마무리** — Session 2~4를 모두 통과했다면 RAG의 5가지 부품, 7가지 고급 기법, 측정·운영까지 완주. 이제 본인 도메인 데이터로 평가셋을 만들고, 본 강의의 측정 코드( `RAGwithVibeCoding/src` )를 출발점 삼아 실제 RAG를 구축하면 된다.